CUHK-X Challenge — Reproduction Package Guidelines

Selection Stage · Small Model Track and Large Model Track · 2026-09-15 v2

We will re-run your final submission on our own machines. Your package should let someone who has never seen your code reproduce your private-leaderboard predictions **without having to contact you**. We do not mind much how the package is organised; what stops a reproduction is **missing information** — an unpinned dependency, an undocumented step, a weight file that is downloaded at run time.

Build your package from the attached **package template** (`CUHK-X_Verification_Package_Template_2026-09-15_v5.zip`; the package is also called the verification package). Its README.md, manifest.yaml and submission/submission_info.yaml ask for everything listed below. These guidelines explain what matters most and how to send the package.

Deadline: 18 September 2026, 23:59 UTC (19 September 2026, 07:59 in UTC+8). The technical report is not part of this package; it is submitted separately.

> **Please leave plenty of time to upload. Bandwidth is limited and packages can be large: a big upload can take many hours, and an upload that fails has to be started again. Start as soon as your package is ready and aim to finish at least 24 hours before the deadline. The submission form closes automatically at the deadline; after that nothing can be submitted or edited.**

1. What the package must contain

- **Code** — the inference code that produces your final Kaggle submission, and your training code (as stated on the competition website). We run inference; the training code is needed for review.
- **Model weights** — every custom weight file loaded at inference time, including each member of an ensemble. Small Model Track packages must follow the track's model-size rule. A public base model need not be included if README.md and manifest.yaml give its exact repository and revision; it may be downloaded during environment setup only — inference itself must not download anything.
- **One entry point** — `inference.sh`, called as `bash inference.sh <test_data_dir> <output_csv>`. It reads the official label-free Testing data in the same internal layout as the published challenge dataset and writes a CSV in the Kaggle submission format. **We will run the same script on new data in the same format in the Organizing Committee's controlled verification environment**, so do not hard-code file lists, sample counts or paths from your own machine. If your two Selected submissions used different models, the same script must also accept the Submission ID as an optional third argument.
- **The environment** — see section 2. This is the most common reason a reproduction fails.
- **README.md, manifest.yaml and submission/submission_info.yaml** — completed from the package template. They tell us your hardware, software versions, the exact commands, how long a run takes and which Kaggle submissions the package reproduces. Section 3 lists what matters most.
- **Honour declaration** — `declarations/CUHK-X_Honor_Declaration.docx`, signed by **every team member**; add a signed PDF copy if you can.

2. The environment: Docker preferred

Option A — Docker image (preferred). Include a `Dockerfile`, and give us the built image either as a `docker save` archive or as a pinned tag on a public registry (Docker Hub, GHCR, …). The image must already contain every dependency and every weight file — nothing may be installed or downloaded when it runs. Set `WORKDIR` to the package root (the directory that holds `inference.sh`). This is the command we will use:

```
docker load -i image.tar.gz           # or: docker pull <registry>/<image>:<tag>
docker run --rm --gpus all --network none --shm-size 16g \
    -v /path/to/test_data:/data:ro \
    -v /path/to/output:/output \
    <image>:<tag>  bash inference.sh /data /output/submission.csv
```

If your image needs a different command, write the exact command in README.md.

Option B — no Docker. Give us a pinned environment: `environment.yml` (from `conda env export`) or `requirements.txt` with exact versions (from `pip freeze`), plus the Python, CUDA and PyTorch versions and any system packages. List the setup commands from a clean Linux machine in README.md.

> Our verification machines run Linux x86_64 with NVIDIA GPUs. If your code needs more than one GPU, more than 24 GB of GPU memory, a specific GPU generation, or does not run on Linux, say so at the top of README.md — early, so we can prepare.

3. What the README must answer

- **Which submissions** it reproduces: every Selected final submission (up to two) — Kaggle submission ID, file name, private score, and the SHA-256 of the exact CSV submitted; mark the one with the higher private score as the primary verification target (the Organizing Committee will confirm it).
- **Needs attention** (section 0 of the template README): more than one GPU, more than 24 GB of GPU memory, a specific GPU generation or driver, network access at inference time, or not Linux — answer these first.
- **Hardware** you ran it on: GPU model and count, peak GPU memory, system RAM, disk space needed.
- **Software**: OS, NVIDIA driver, CUDA, Python and PyTorch versions — or the Docker image name and tag.
- **Commands**: setup from scratch, then the exact inference command, and where the output is written.
- **Run time** of one full inference over the test set on your hardware.
- **Internet at run time**: none, or which service is called and why.
- **Randomness**: seeds used; whether the output is exactly reproducible; if not, how much the score varies.
- **Files**: which checkpoint files are used, their sizes, and anything unusual (multi-stage pipelines, ensembles, caches that must be rebuilt).

4. Closed-source APIs (Large Model Track)

If inference calls a closed-source API, read the key from an environment variable (for example `OPENAI_API_KEY`) and **never put your own key in the package**. In README.md, state which service and model version you call, roughly how many calls one full run makes and what it costs, and whether results are cached. We will contact API-based teams about how their verification will be run.

5. Package layout (from the template)

```
small_or_large_TEAMNAME_verification_v1/     ← from the package template
├── README.md               ← start here: sections 0–9
├── manifest.yaml           ← every artifact, the environment, network and API use
├── inference.sh            ← the single entry point
├── src/  configs/          ← your inference (and training) code and configuration
├── artifacts/              ← custom weights, adapters, prompts
├── environment/            ← Dockerfile, and/or environment.yml / requirements.txt
├── submission/             ← submission_info.yaml + the Kaggle CSV(s) of every Selected
submission
└── declarations/           ← honour declaration signed by every team member

image.tar.gz                ← optional separate file: docker save output
```

Please **do not include the competition dataset** — we use our own copy.

6. Before you send it: a five-minute self-check

The best way to find a missing piece is to reproduce your own result the way we will. On a fresh machine or container, with the network off, run the documented command and compare the output with the CSV you submitted to Kaggle:

```
# on a fresh machine or a fresh container, with the network switched off:
docker run --rm --gpus all --network none --shm-size 16g \
    -v $PWD/test_data:/data:ro -v $PWD/out:/output <image>:<tag> \
    bash inference.sh /data /output/submission.csv

# then compare with the CSV you submitted to Kaggle
python -c "import pandas as p; a=p.read_csv('out/submission.csv');
b=p.read_csv('kaggle_submission.csv'); print(a.equals(b))"
```

7. Packing and sending

```
# one archive, split into parts of at most 9 GB if it is larger than 10 GB
# 7-Zip, any OS:
7z a -v9g large_myteam_verification_v1.7z large_myteam_verification_v1/
# Linux / macOS:
tar czf - large_myteam_verification_v1 | split -b 9G -d -
large_myteam_verification_v1.tar.gz.part-

# Docker image as a file
docker save myimage:v1 | gzip > image.tar.gz

# checksums to paste into the form
sha256sum *            # Linux     |  shasum -a 256 *  (macOS)
Get-FileHash -Algorithm SHA256 * | Format-List        # Windows PowerShell
```

- **Upload in the submission form** — up to 10 files per submission, 10 GB per file. Split larger archives into parts of at most 9 GB. You can edit your submission until the deadline, when the form closes; submit the form again only if you need to send more than 10 files.
- **Or share a download link** in the form (Google Drive, OneDrive, Dropbox, Hugging Face, Baidu Netdisk or any HTTPS link), or attach small files to an email.
- **If none of these works for you**, email cuhkx.competition@gmail.com and we will work it out together.
- If your team is in both tracks, send one package per track.

If your team selected two final submissions on Kaggle, disclose both. Your package must reproduce the one with the higher private score — the primary verification target. Enter it yourself; the Organizing Committee will confirm it against Kaggle's records.

Questions about these guidelines: cuhkx.competition@gmail.com